

TUGAS UAS
PEMROGRAMAN BERORIENTASI OBYEK

Dosen Pengampu:
ZAINAL ABIDIN,S.Kom,M.Kom



Disusun Oleh:
Rahmat Enomoto (230605110010)

PROGRAM STUDI
TEKNIK INFORMATIKA
FAKULTAS SAINS DAN TEKNOLOGI
UNIVERSITAS ISLAM NEGERI MAULANA MAILK IBRAHIM MALANG
2024

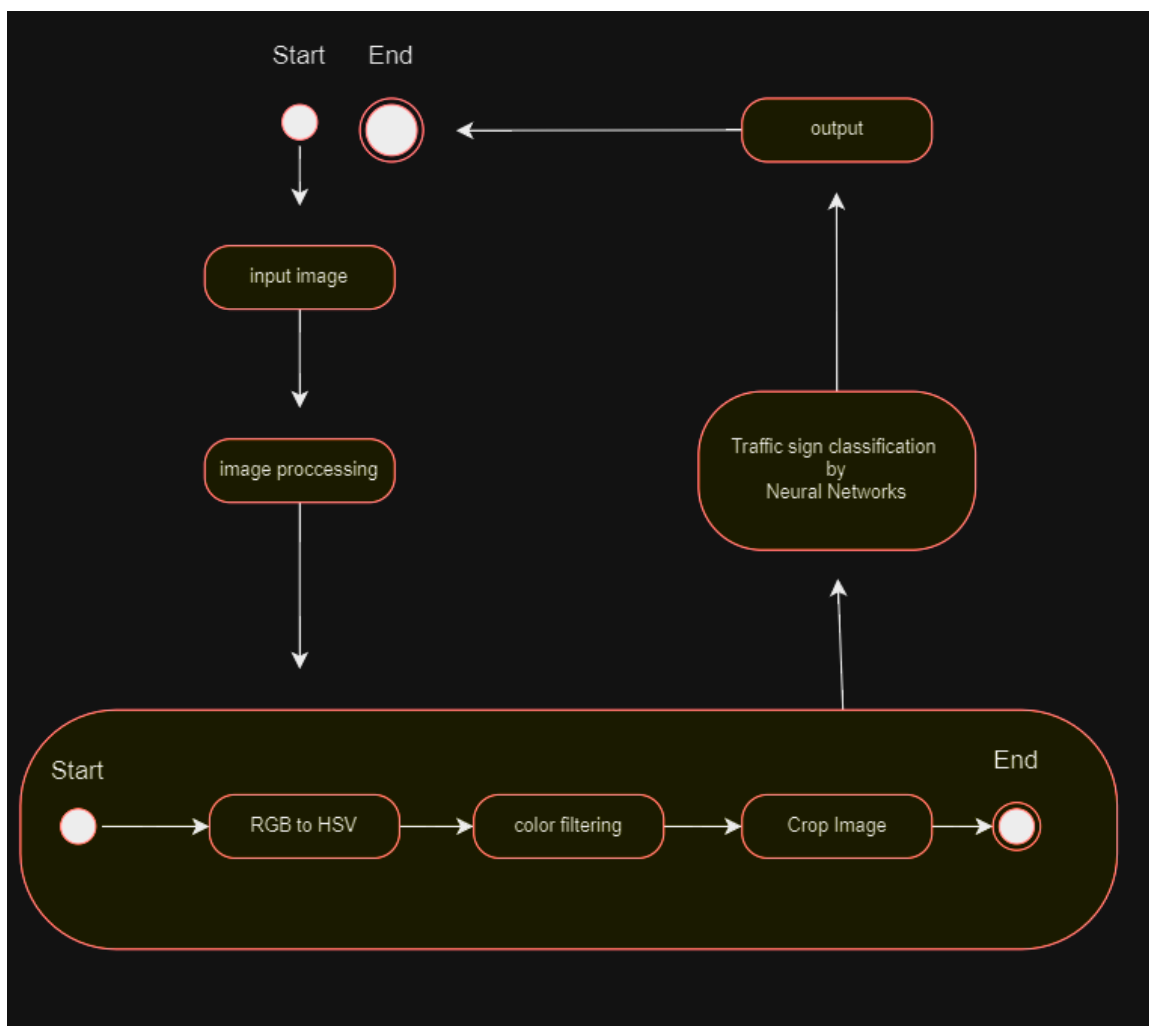
Traffic Sign Detection and Classification

I. PENDAHULUAN

Traffic Sign detection merupakan teknologi yang penting dalam pengembangan navigasi pada kendaraan otomatis. Dalam era dimana kendaraan otomatis semakin berkembang pesat. Kemampuan untuk mengenali dan memahami rambu-rambu lalu lintas dengan akurat sangat diperlukan untuk keamanan dan efisiensi transportasi.

Rambu lalu lintas memberikan informasi penting kepada pengemudi dan sistem navigasi mengenai batas kecepatan, arah, kondisi jalan dan peringatan lainnya. Oleh karena itu teknologi Traffic Sign Detection yang canggih sangat diperlukan dalam mengembangkan teknologi kendaraan otonom.

II. OVERVIEW OF PROPOSED SYSTEM



Gambar di atas menunjukkan 5 tahapan utama: Image processing, konversi rgb ke hsv, color filtering, Crop image, dan Trafficsign classification menggunakan neural network.

III. TRAFFIC SIGN DETECTION

Tahap ini terdiri dari image processing (meningkatkan kualitas citra), Konversi RGB ke HSV, Color filtering dan Crop Image

A. Meningkatkan kualitas citra

Meningkatkan kualitas citra untuk traffic sign adalah proses yang bertujuan untuk memperjelas dan memperbaiki aspek tampilan sehingga lebih mudah saat dieksekusi program.

- Pertama: Teknik histogram equalization digunakan untuk meningkatkan kontras gambar dengan meratakan distribusi intensitas piksel, sehingga detail pada rambu lalu lintas menjadi lebih menonjol.
- Gaussian blur membantu mengurangi noise yang dapat mengganggu proses deteksi dengan menghaluskan variasi intensitas yang tidak diinginkan.
- teknik unsharp masking diterapkan untuk mempertegas tepi-tepi dan kontur pada gambar, sehingga bentuk dan warna rambu lalu lintas lebih jelas terlihat

B. Konversi RGB ke HSV

Mendeteksi Rambu lalu lintas menggunakan model RGB memiliki beberapa kelemahan yaitu:

- Sensivitas Terhadap Pencahayaan: Pada saat bayangan, cahaya terang, atau perubahan waktu siang dan malam dapat mempengaruhi akurasi deteksi. Sistem berbasis RGB akan sedikit kurang akurat dan sulit menyesuaikan perubahan pencahayaan.
- Gangguan Warna Latar Belakang: Warna latar belakang yang mirip dengan warna rambu lalu lintas dapat menyebabkan kesulitan dalam pemisahan objek dan latar. Hal ini terutama terjadi di lingkungan yang kompleks dan bervariasi.
- Keterbatasan Resolusi: Kamera dengan resolusi rendah mungkin tidak mampu menangkap detail warna yang cukup, terutama ketika rambu lalu lintas kecil atau berada jauh dari kamera.

- Ketidakstabilan Warna: Warna yang ditangkap oleh kamera bisa bervariasi tergantung pada kualitas kamera, kalibrasi, dan kondisi atmosfer (misalnya kabut atau hujan), sehingga warna yang terdeteksi mungkin tidak konsisten.

Untuk mengatasi kelemahan-kelemahan ini, teknik tambahan seperti penggunaan model HSV (Hue, Saturation, Value) yang lebih stabil terhadap perubahan pencahayaan. Dalam Program model warna yang ditangkap oleh program biasanya menggunakan RGB jadi kita harus mengkonversi warna dari RGB ke HSV.

```
package AA_OOP_TSD;

public class rumusRGBtoHSV {
    private int red;
    private int green;
    private int blue;

    public rumusRGBtoHSV() {
    }

    public rumusRGBtoHSV(int red, int green, int blue) {
        this.red = red;
        this.green = green;
        this.blue = blue;
    }

    public int getRed() {
        return red;
    }

    public void setRed(int red) {
        this.red = red;
    }

    public int getGreen() {
        return green;
    }

    public void setGreen(int green) {
        this.green = green;
    }

    public int getBlue() {
        return blue;
    }

    public void setBlue(int blue) {
        this.blue = blue;
    }

    public double[] convert() {
        double r = red / 255.0;
        double g = green / 255.0;
        double b = blue / 255.0;
        double[] rgb = {r, g, b};
```

```
        double min = rgb[0];
        double max = rgb[0];
        for (int i = 1; i < rgb.length; i++) {
            if (min > rgb[i]) {
                min = rgb[i];
            }
            if (max < rgb[i]) {
                max = rgb[i];
            }
        }
        double delta = max - min;
        double hue = 0;
        if (delta != 0) {
            if (max == r) {
                hue = 60 * ((g - b) / delta) % 6;
            } else if (max == g) {
                hue = 60 * ((b - r) / delta) + 2;
            } else {
                hue = 60 * ((r - g) / delta) + 4;
            }
        }
        if (hue < 0) {
            hue += 360;
        }
        double saturation = (max == 0) ? 0 : (delta / max);
        double value = max;
        return new double[]{hue, saturation, value};
    }
}
```

Class `rumusRGBtoHSV` berfungsi sebagai class untuk mengkonversi nilai warna dari ruang warna RGB (Red, Green, Blue) ke ruang warna HSV (Hue, Saturation, Value).

- Yang pertama mengkonversi nilai red, green, dan blue yang diberikan sebagai parameter dikonversi ke nilai antara 0 dan 1 dengan membagi masing-masing dengan 255

```
public double[] convert() {  
    double r = red / 255.0;  
    double g = green / 255.0;  
    double b = blue / 255.0;  
}
```

- Mencari nilai minimum dan maximum untuk menentukan nilai delta

```
double[] rgb = {r, g, b};  
double min = rgb[0];  
double max = rgb[0];  
for (int i = 1; i < rgb.length; i++) {  
    if (min > rgb[i]) {  
        min = rgb[i];  
    }  
    if (max < rgb[i]) {  
        max = rgb[i];  
    }  
}  
double delta = max - min;
```

- Menghitung Hue: dihitung berdasarkan nilai maksimum dan perbedaan antara warna yang berbeda. Jika delta tidak sama dengan nol, hue dihitung dengan rumus yang berbeda tergantung pada warna mana yang maksimum

```
double hue = 0;  
if (delta != 0) {  
    if (max == r) {  
        hue = 60 * ((g - b) / delta) % 6;  
    } else if (max == g) {  
        hue = 60 * ((b - r) / delta) + 2;  
    } else {  
        hue = 60 * ((r - g) / delta) + 4;  
    }  
}  
if (hue < 0) {  
    hue += 360;  
}
```

- Menghitung saturasi: Saturation dihitung sebagai perbandingan antara delta dan max. Jika max sama dengan nol, saturation diatur ke nol.

```
double saturation = (max == 0) ? 0 : (delta / max);
```

- Menghitung Value: Nilai Value sama dengan nilai maksimum dari (r/g/b).

```
double value = max;
```

- Return value `return new double[]{hue, saturation, value};`

Konsep oop yang di gunakan

- Enkapsulasi: Bertujuan untuk mengamankan data dan mencegah modifikasi yang

tidak sah

```
private int red;  
private int green;  
private int blue;
```

```
public int getRed() {  
    return red;  
}  
  
public void setRed(int red) {  
    this.red = red;  
}  
  
public int getGreen() {  
    return green;  
}  
  
public void setGreen(int green) {  
    this.green = green;  
}  
  
public int getBlue() {  
    return blue;  
}  
  
public void setBlue(int blue) {  
    this.blue = blue;  
}
```

- Konstruktor:

```
public rumusRGBtoHSV(int red, int green, int blue) {  
    this.red = red;  
    this.green = green;  
    this.blue = blue;  
}
```

C. Color Filtering

Color Filtering merupakan suatu metode yang bertujuan untuk menampilkan warna yang diinginkan dan merubah/menekan warna lainnya.

Metode Color Filtering

- Seleksi Warna: Memilih rentang warna tentu

```
package AA_OOP_TSD;  
  
public class seleksiWarna {  
    public static boolean isRed(double[] hsv) {  
        return (hsv[0] >= 0 && hsv[0] <= 10 || hsv[0] >= 350 && hsv[0] <= 360) && hsv[1] >= 0.5 && hsv[2] >= 0.5;  
    }  
    public static boolean isBlue(double[] hsv) {  
        return hsv[0] >= 180 && hsv[0] <= 240 && hsv[1] >= 0.5 && hsv[2] >= 0.3;  
    }  
    public static boolean isYellow(double[] hsv) {  
        return hsv[0] >= 10 && hsv[0] <= 65 && hsv[1] >= 0.7 && hsv[2] >= 0.75;  
    }  
    public static boolean isGreen(double[] hsv) {  
        return hsv[0] >= 85 && hsv[0] <= 140 && hsv[1] >= 0.5 && hsv[2] >= 0.3;  
    }  
    public static boolean isOrange(double[] hsv) {  
        return hsv[0] >= 15 && hsv[0] <= 40 && hsv[1] >= 0.5 && hsv[2] >= 0.5;  
    }  
}
```

Pada class ini berisi method memeriksa apakah nilai hue (H), saturasi (S), dan nilai (V) berada dalam rentang yang ditentukan untuk warna tertentu. Misalnya, untuk warna merah,

hue harus berada antara 0 dan 10 atau antara 350 dan 360, dan saturasi serta nilai harus lebih dari atau sama dengan 0.5.

- Color Thresholding

Thresholding merupakan teknik di mana menentukan batas-batas (thresholds) untuk masing-masing komponen warna (misalnya, dalam model HSV: hue, saturation, value) dan mengubah piksel-piksel yang berada di luar batas-batas ini menjadi hitam. Sementara yang berada di dalam batas-batas ini masih tetap. Misalnya, untuk mendeteksi warna merah pada rambu lalu lintas, Kita menentukan rentang nilai hue, saturation, dan value dalam model HSV yang mewakili warna merah. Piksel-piksel yang tidak berada dalam rentang ini kemudian diubah menjadi hitam.

```
package AA_OOP_TSD;

import java.awt.image.BufferedImage;
import java.io.File;
import javax.imageio.ImageIO;

public class ImageProcessor {
    private BufferedImage image;

    public ImageProcessor(String imagePath) throws Exception {
        this.image = ImageIO.read(new File(imagePath));
    }

    public void thresholding() {
        int width = image.getWidth();
        int height = image.getHeight();
        BufferedImage filteredImage = new BufferedImage(width, height, BufferedImage.TYPE_INT_RGB);

        for (int x = 0; x < width; x++) {
            for (int y = 0; y < height; y++) {
                int rgb = image.getRGB(x, y);
                int red = (rgb >> 16) & 0xFF;
                int green = (rgb >> 8) & 0xFF;
                int blue = rgb & 0xFF;

                rumusRGBtoHSV rgbHSV = new rumusRGBtoHSV();
                rgbHSV.setRed(red);
                rgbHSV.setGreen(green);
                rgbHSV.setBlue(blue);
                double[] hsv = rgbHSV.convert();
```

```
                if (seleksiWarna.isRed(hsv) || seleksiWarna.isBlue(hsv) || seleksiWarna.isYellow(hsv) || seleksiWarna.isGreen(hsv) ||
                    seleksiWarna.isOrange(hsv)) {
                    filteredImage.setRGB(x, y, rgb);
                } else {
                    filteredImage.setRGB(x, y, rgb:0x000000);
                }
            }
        }
        this.image = filteredImage;
    }

    public void saveImage(String outputPath) throws Exception {
        ImageIO.write(image, formatName:"jpg", new File(outputPath));
    }
}
```

Class diatas berfungsi untuk mengubah piksel-piksel yang berada di luar batas-batas ini menjadi hitam. Sementara yang berada di dalam batas-batas ini masih tetap.

1. Deklarasi Class ImageProcessor

- `BufferedImage image`: Variabel yang menyimpan citra sebagai objek `BufferedImage`.
- Kontruktor `ImageProcessor (String imagePath)`: Menerima path gambar sebagai argumen, membaca gambar dari file yang sesuai, dan menyimpannya dalam `BufferedImage`.

```
public class ImageProcessor {  
    private BufferedImage image;  
  
    public ImageProcessor(String imagePath) throws Exception {  
        this.image = ImageIO.read(new File(imagePath));  
    }  
}
```

2. Method Thresholding

- `width` dan `height`: Variabel yang menyimpan lebar dan tinggi citra.
- `BufferedImage filteredImage`: Objek `BufferedImage` baru untuk menyimpan citra hasil proses.

```
public void thresholding() {  
    int width = image.getWidth();  
    int height = image.getHeight();  
    BufferedImage filteredImage = new BufferedImage(  
        width, height, BufferedImage.TYPE_INT_RGB);  
}
```

- Dua perulangan `for nested` digunakan untuk mengiterasi setiap piksel dalam citra (mengambil nilai `rgb`)
 - `int rgb = image.getRGB(x, y);`: Mengambil nilai RGB piksel pada koordinat `(x, y)`.
 - Bit 24-31: Alpha (transparansi), Bit 16-23: Red (merah), Bit 8-15: Green (hijau), Bit 0-7: Blue (biru)
 - `int red = (rgb >> 16) & 0xFF`: Mendapatkan komponen warna merah dengan operasi bitwise.
 - `int green = (rgb >> 8) & 0xFF`: Mendapatkan komponen warna hijau dengan operasi bitwise.
 - `int blue = rgb & 0xFF`: Mendapatkan komponen warna biru dengan operasi bitwise.


```
for (int x = 0; x < width; x++) {
    for (int y = 0; y < height; y++) {
        int rgb = image.getRGB(x, y);
        int red = (rgb >> 16) & 0xFF;
        int green = (rgb >> 8) & 0xFF;
        int blue = rgb & 0xFF;
    }
}
```

- Konversi ke HSV

- `rumusRGBtoHSV rgbHSV = new rumusRGBtoHSV();`: Membuat objek `rumusRGBtoHSV` untuk konversi ke nilai HSV.

```
rumusRGBtoHSV rgbHSV = new rumusRGBtoHSV();
```

- `rgbHSV.setRed(red)`, `rgbHSV.setGreen(green)`, `rgbHSV.setBlue(blue)`: Mengatur nilai komponen warna pada objek `rumusRGBtoHSV`.

```
rgbHSV.setRed(red);
rgbHSV.setGreen(green);
rgbHSV.setBlue(blue);
```

- `double [] hsv = rgbHSV.convert();`: Memanggil metode `convert()` dari class `rumusRGBtoHSV` untuk mengonversi nilai RGB menjadi nilai HSV.

```
double[] hsv = rgbHSV.convert();
```

- Penerapan thresholding

- Menggunakan kelas `seleksiWarna` untuk menentukan apakah nilai HSV memenuhi kriteria tertentu (`isRed()`, `isBlue()`, dst.).
- Jika memenuhi kriteria, warna piksel tetap dipertahankan.
- Jika tidak memenuhi kriteria, warna piksel diubah menjadi hitam (`0x000000`).

```
if (seleksiWarna.isRed(hsv) || seleksiWarna.isBlue(hsv) || seleksiWarna.isYellow(hsv) || seleksiWarna.isGreen(hsv) ||
    seleksiWarna.isOrange(hsv)) {
    filteredImage.setRGB(x, y, rgb);
} else {
    filteredImage.setRGB(x, y, rgb:0x000000);
}
```

3. Method Save Image

- `saveImage(String outputPath)`: Metode untuk menyimpan citra hasil proses ke dalam file JPG.
- `ImageIO.write(image, "jpg", new File(outputPath))`: Menulis citra yang disimpan dalam `BufferedImage image` ke dalam file dengan format JPG sesuai dengan path yang diberikan.

```
public void saveImage(String outputPath) throws
Exception {
    ImageIO.write(image, formatName:"jpg", new File
(outputPath));
}
```

Konsep oop yang digunakan

- Enkapsulasi: `private BufferedImage image;`
- Konstrktor:

```
public class ImageProcessor {
    private BufferedImage image;

    public ImageProcessor(String imagePath) throws
Exception {
        this.image = ImageIO.read(new File(imagePath));
    }
}
```

D. Crop Image

Resize image merupakan proses penting dalam Traffic Sign detection, Bertujuan untuk meningkatkan akurasi dan efisiensi dalam proses Traffic Sign Detection and Classification. Crop membantu memperbesar dan memfokuskan area yang ingin di analisis, Dengan memotong bagian yang tidak perlu maka hal itu dapat mempercepat waktu pemrosesan dan meningkatkan akurasi dalam klasifikasi. Selain itu Crop Image juga membantu mengurangi beban komputasi.

Alur Proses Crop Image:

- Deteksi Koordinat Rambu Lalu Lintas: Setelah proses color filtering dalam model HSV, Kita menggunakan gambar hasil proses color filtering. Langkah pertama dalam crop image adalah menentukan koordinat dari rambu lalu lintas yang telah terdeteksi dalam gambar.
- Penentuan Bounding Box: Dari hasil deteksi rambu lalu lintas dalam gambar, Gambar memiliki bounding box yang menunjukkan area yang mengelilingi rambu lalu lintas. Bounding box ini berupa koordinat (misalnya, titik pojok atas kiri dan titik pojok bawah kanan) yang menentukan batas-batas dari rambu lalu lintas tersebut.

- Crop Gambar Berdasarkan Bounding Box: Gunakan koordinat dari bounding box untuk memotong bagian gambar yang mengandung rambu lalu lintas.

IV. TRAFFIC SIGN CLASSIFICATION

Traffic sign classification Bertujuan untuk mengklasifikasi Trafficsign Metode yang umum digunakan untuk traffic sign classification melibatkan ekstraksi fitur dari gambar, seperti bentuk, warna, dan pola, yang kemudian digunakan sebagai masukan untuk model klasifikasi seperti neural networks.

Alur Traffic sign classification menggunakan neural network:

1. Pengumpulan dan Praproses Data:

Data gambar tanda lalu lintas dikumpulkan dari berbagai sumber atau direkam menggunakan kamera.

Setiap gambar perlu diproses sebelum dimasukkan ke dalam model neural network, seperti resizing ke ukuran yang sama, normalisasi intensitas piksel, dan mungkin augmentasi data untuk meningkatkan keberagaman dataset.

2. Pembagian Dataset:

Dataset dibagi menjadi set pelatihan (training set), validasi (validation set), dan pengujian (test set). Setiap set berisi gambar tanda lalu lintas beserta labelnya (jenis tanda lalu lintas).

3. Pemilihan Model Neural Network:

Pilih arsitektur neural network yang sesuai untuk masalah klasifikasi gambar, seperti Convolutional Neural Network (CNN). CNN sering digunakan karena mampu mengekstrak fitur dari gambar dengan baik.

4. Pelatihan Model:

Model neural network dilatih menggunakan data pelatihan. Proses ini melibatkan iterasi di mana model memperbarui bobotnya (weights) berdasarkan kesalahan prediksi terhadap data pelatihan.

Optimizer seperti Stochastic Gradient Descent (SGD), Adam, atau RMSprop digunakan untuk menyesuaikan bobot model.

5. Validasi Model:

Selama pelatihan, model dievaluasi menggunakan data validasi untuk memonitor kinerjanya dan mencegah overfitting.

Metrik seperti akurasi (accuracy), presisi (precision), recall, dan F1-score digunakan untuk mengevaluasi kinerja model.

6. Evaluasi Model:

Setelah pelatihan selesai, model dievaluasi menggunakan data pengujian yang tidak pernah dilihat sebelumnya untuk mengukur kinerjanya pada data baru yang independen.

7. Penyetelan Hyperparameter (Opsional):

Jika diperlukan, hyperparameter seperti learning rate, jumlah epoch, dan ukuran batch dapat disesuaikan untuk meningkatkan kinerja model.

8. Prediksi dan Deploy Model:

Setelah model dilatih dan dievaluasi dengan baik, model tersebut siap digunakan untuk memprediksi kelas tanda lalu lintas dari gambar baru yang masuk.

9. Monitoring dan Pemeliharaan:

Model perlu dipantau secara teratur setelah dideploy untuk memastikan kinerjanya tetap optimal. Perlu dilakukan pemeliharaan jika kinerja model menurun atau jika ada perubahan dalam sumber data.

V. HASIL DAN PEMBAHASAN

Setelah perobaan membuat program Traffic sign detection sederhana selama satu semester, Saya baru sampai pada tahap Traffic sign detection tepat nya pada metode Color Filtering

Ini adalah beberapa hasil dari Traffic sign detection:



filtered_image.jpg





VI. KESIMPULAN

Dalam pengembangan teknologi kendaraan otonom, kemampuan untuk mendeteksi dan mengklasifikasi rambu lalu lintas dengan akurat sangat diperlukan untuk keamanan dan efisiensi transportasi.

Tahap utama dalam sistem meliputi Image processing, Konversi RG ke HSV, Color filtering, Crop image dan klasifikasi rambu lalu lintas menggunakan neural network.

Dalam proses deteksi penggunaan model waena HSV terbukti lebih efektif dibandingkan model RGB.